Research methodology practical file

Name-Piyush Jha

Roll No.-1623048

Semester- VI

Session-2025-26

Submitted to: Dr. Rakhi Narang

Submitted by: Piyush

Index

S.no.	Name of experiment	Remarks
1.	Hypothesis testing on a real time dataset	
2.	Latex software usage for review article: Including Abstract , Creating heading/sub-headings, Adding images and tables with captions, adding mathematical equations, including citations.	
3.	Curve fitting and Interpolation using Excel /python /origin	
4.	Reference Management using MS Word/EndNote/Mendeley	
5.	Secondary data plotting and analysis of given data set using Origin.	
6.	Applications of AI driven apps like Litmap, explain paper, open access plagiarism tools for research work	
7.	Writing a Review Article	

Experiment-1

Aim: Hypothesis testing on a real time dataset.

Software used: MS Excel

Theory: Hypothesis testing is a statistical method used to make decisions or draw conclusions about a population based on sample data. It helps determine whether an observed effect or result is significant or occurred by chance.

In this process, two hypotheses are defined:

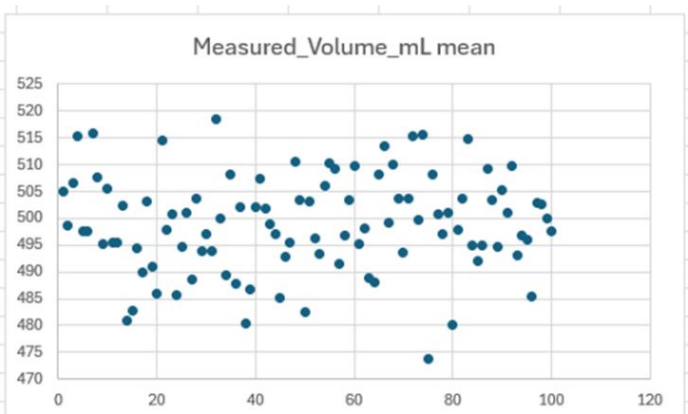
- **Null Hypothesis (H_0):** Assumes no effect or no difference
- **Alternative Hypothesis (H_1):** Assumes there is an effect or difference

Data is analyzed using statistical tests (like t-test, z-test, etc.), and a **p-value** is calculated.

- If **p-value < significance level (α)** → reject H_0
- If **p-value ≥ α** → fail to reject H_0

Hypothesis testing is widely used in research to validate assumptions, compare datasets, and support scientific conclusions with statistical evidence.

Data Plot:



Calculated Values:

x(mean)	500
u	498.9615
root of n	10
std	9.081684
z	500-498.9615/0.9081684
	1.1435
p value	0.8729

Inference:

The calculated p-value (0.8729) is greater than the significance level, so the null hypothesis is not rejected.
This indicates that there is no significant difference between the sample mean and the assumed population mean, and the variation is likely due to random fluctuations.

Experiment-2

Aim :- Latex software usage for review article: Including Abstract, creating Heading/sub-headings, Adding images and tables with captions, adding mathematical equations, including citations.

Hypothesis Testing

Piyush Jha

April 12, 2026

Abstract

Hypothesis testing is a statistical method used to make decisions using data. This article explains types of hypotheses, testing steps, and a numerical example.

1 Introduction to Hypothesis Testing

Hypothesis testing is a fundamental concept in statistics used to make inferences about a population based on sample data. It helps determine whether observed differences are statistically significant.

2 Types of Hypothesis

2.1 Null Hypothesis (H_0)

States no effect or no difference.

Example: $H_0 : \mu = 203$

2.2 Alternative Hypothesis (H_1)

Indicates presence of effect.

Example: $H_1 : \mu \neq 203$

2.3 Directional vs Non-directional

Directional specifies direction; non-directional does not.

3 Steps of Hypothesis Testing

1. Formulation of Hypotheses
2. Selection of Significance Level (α)
3. Selection of Test Statistic

4. Compute Test Statistic
5. Find p-value
6. Decision Making
7. Conclusion

4 Flowchart Representation

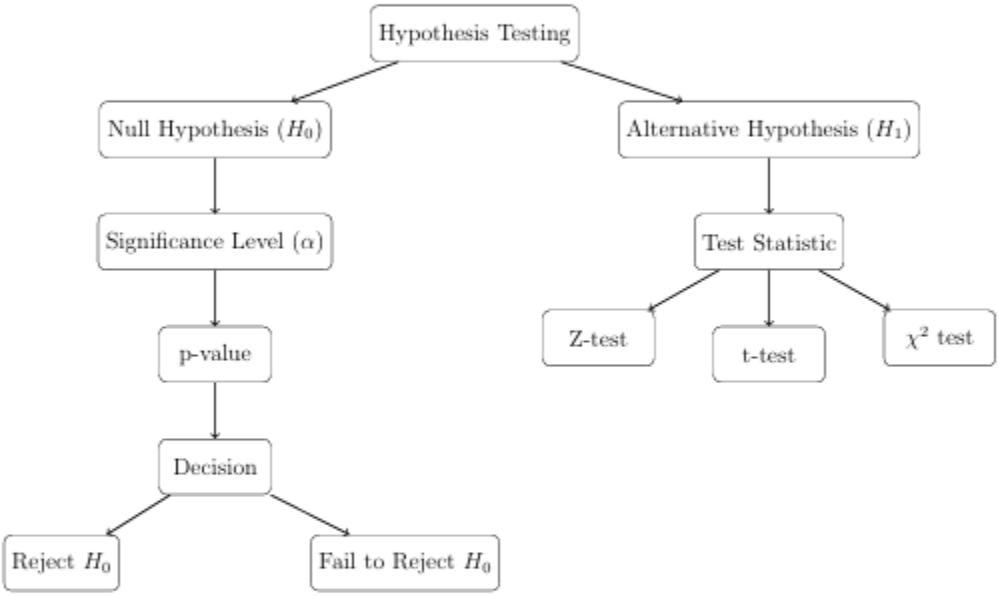


Figure 1: Flowchart of Hypothesis Testing

5 Numerical Problem

Given:

$$\mu = 203, \quad \bar{x} = 200, \quad s = 36.8, \quad n = 3310, \quad \alpha = 0.05$$

Step 1: Hypotheses

$H_0 : \mu = 203$
$H_1 : \mu \neq 203$

Step 2: Test Statistic

Since $n > 30$, Z-test is used.
:contentReference[oaicite:1]index=1

Step 3: Calculation

$$Z = \frac{200 - 203}{36.8/\sqrt{3310}} = -4.69$$

Step 4: p-value

$$p \approx 0.0000$$

Step 5: Decision

Since $p < 0.05$, reject H_0.

Step 6: Conclusion

The mean differs significantly from 203 mg/dL.

6 Summary Table

Type	Form	Use
Null (H_0)	$\mu = \mu_0$	No effect
Alt (Two-tailed)	$\mu \neq \mu_0$	Any difference
Alt (Right-tailed)	$\mu > \mu_0$	Greater than
Alt (Left-tailed)	$\mu < \mu_0$	Less than

Table 1: Types of Hypothesis

7 Summary

Hypothesis testing is essential in statistical decision-making. This study demonstrated its application using a Z-test.

Experiment-3

Aim: Curve fitting and Interpolation using Excel/python/origin

Theory:

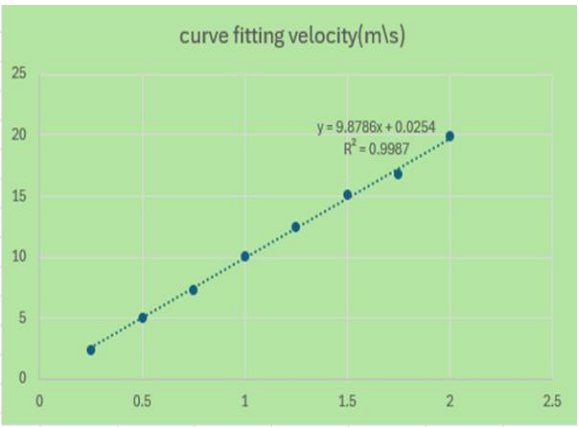
Curve fitting is a method used to find a mathematical function that best represents a set of data points. It helps in identifying trends and relationships between variables.

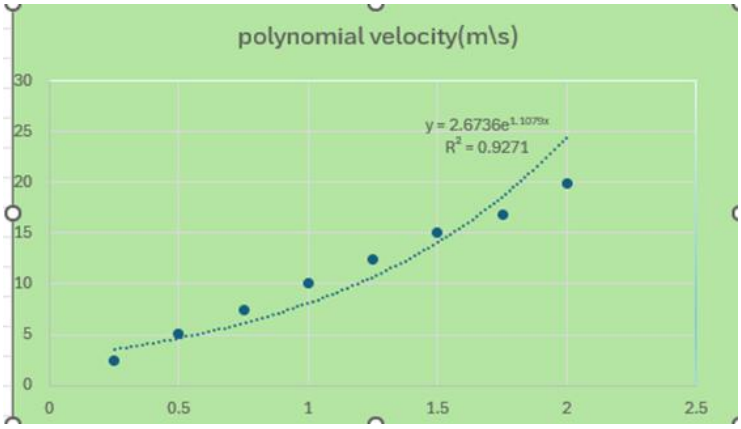
Common methods include linear and polynomial fitting, often using the least squares technique. It is widely used to model data and make predictions in scientific analysis.

Dataset:

time(s)	velocity(m\s)	position(m)
0.25	2.39	0.3
0.5	5.02	1.22
0.75	7.36	2.67
1	10.04	4.82
1.25	12.44	7.54
1.5	15.09	10.66
1.75	16.85	14.99
2	19.92	19.44

Plots:





Interpolation:

Interpolation is a technique used to estimate unknown values within the range of known data points. It involves constructing a function that passes through the given data to predict intermediate values.

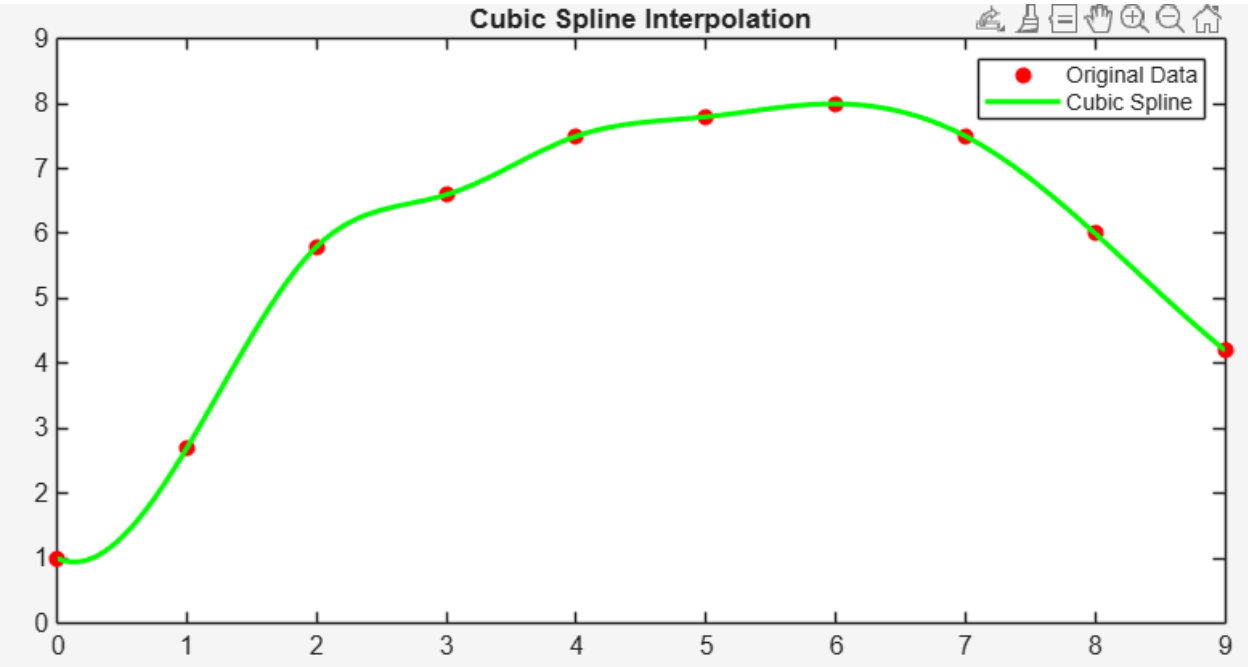
Common methods include linear and polynomial interpolation. It is widely used in scientific and engineering applications for data estimation and analyzing trends in experimental data.

Code:

```
x = [0 1 2 3 4 5 6 7 8 9];
y = [1.0 2.7 5.8 6.6 7.5 7.8 8.0 7.5 6.0 4.2];
x_dense = linspace(min(x), max(x), 200

% Polynomial Curve Fitting
p = polyfit(x, y, 3);
y_poly = polyval(p, x_dense);
figure;
plot(x, y, 'ro', 'MarkerFaceColor', 'r');
hold on;
plot(x_dense, y_poly, 'b-', 'LineWidth', 2);
title('Polynomial Curve Fitting (Degree 3)');
legend('Original Data', 'Polynomial Fit');

% Cubic Spline Interpolation
y_spline = spline(x, y, x_dense);
figure;
plot(x, y, 'ro', 'MarkerFaceColor', 'r');
hold on;
plot(x_dense, y_spline, 'g-', 'LineWidth', 2);
title('Cubic Spline Interpolation');
legend('Original Data', 'Cubic Spline');
```



Experiment-4

Aim: Reference Management using MS Word/EndNote/Mendeley.

Article:

Network Slicing for 5G and Beyond 5G Networks

Abstract:

The transition to fifth-generation (5G) and beyond 5G (B5G) networks marks a paradigm shift from a "one-size-fits-all" architecture to a highly flexible, service-oriented ecosystem. At the core of this transformation is network slicing, a key technology that enables the division of a single physical infrastructure into multiple logical, independent networks tailored to specific service requirements. This article provides a comprehensive overview of network slicing, exploring its architectural foundations, the critical role of Software-Defined Networking (SDN) and Network Function Virtualization (NFV), and its realization in diverse Internet of Things (IoT) and smart city applications. We discuss the technical challenges associated with resource management, mobility, and security, and highlight future research directions including Artificial Intelligence (AI) and blockchain integration.

1. Introduction

The exponential growth of handheld devices and the emergence of data-intensive applications like virtual and augmented reality have driven the need for 5G networks. These networks are projected to support billions of connected devices, necessitating a highly dynamic and flexible infrastructure. Traditional monolithic architectures are insufficient for the diverse quality-of-service (QoS) demands of modern verticals. Network slicing addresses this by creating end-to-end (E2E) logical networks that satisfy heterogeneous networking demands through dedicated slices.

2. Architecture and Enabling Technologies

A functional network slice typically consists of two primary subslices: the Radio Access Network (RAN) subslice and the core network subslice. These layers are orchestrated to provide seamless E2E connectivity.

2.1 SDN and NFV

- **SDN (Software-Defined Networking):** Offers centralized control and network programmability by separating the control plane from the data plane.
- **NFV (Network Function Virtualization):** Decouples network functions from dedicated hardware, allowing them to run as software on commodity servers.
- **MEC (Multi-access Edge Computing):** Minimizes backhaul congestion and reduces latency by providing cloud computing capabilities at the network edge.

2.2 Network Slice Lifecycle

The lifecycle of a network slice instance (NSI) includes four phases: preparation (design and onboarding), commissioning (creation and resource allocation), operation (activation and monitoring), and decommissioning (reclamation of resources).

3. Realization in IoT and Smart Cities

Network slicing is a critical enabler for various application scenarios with distinctive requirements.

- **eMBB (Enhanced Mobile Broadband):** Focuses on high-data-rate services such as HD video streaming.
- **mMTC (Massive Machine Type Communications):** Supports a high density of devices, typical in smart home and smart city sensor networks.
- **URLLC (Ultra-Reliable Low-Latency Communications):** Vital for mission-critical applications like autonomous driving and remote surgery.

3.1 Smart City Applications

In smart cities, network slicing supports emergency response through rescue drones and holographic communication, requiring sub-millisecond latency and ultra-high reliability. Industrial IoT (IIoT) applications utilize dedicated slices for factory automation and remote monitoring to increase efficiency and reduce costs.

4. Resource Management and Challenges

Efficient resource allocation is paramount to satisfy conflicting objectives such as energy consumption, delay, and distortion.

- **Auction-Based Models:** Competitive mechanisms can optimize resource allocation to maximize operator revenue while satisfying slice requirements.
- **Mobility Management:** New schemes are required to guarantee seamless handovers in diverse 5G scenarios, particularly for high-speed mobility.
- **Isolation and Security:** Slice isolation is essential to mitigate security breaches (e.g., DDoS attacks) and ensure the privacy of sensitive data in vertical industries.

5. Future Directions

The integration of AI and Machine Learning (AI/ML) is expected to provide pervasive network intelligence, enabling autonomous resource orchestration and predictive traffic demand analysis. Furthermore, blockchain technology is being explored to allow secure creation of slices and decentralized resource management among diverse stakeholders.

Conclusion

Network slicing is the cornerstone of the 5G and B5G vision, providing the scalability and flexibility needed for the next generation of mobile services. While significant progress has been made in standardizing its architecture, ongoing research into intelligent management, enhanced security, and multi-domain orchestration remains critical for its full-scale realization

Citations:

(Abdellatif et al., 2023; Jiang et al., 2017; Rafique et al., 2024; Rost et al., 2017; Subedi et al., 2021; Wijethilaka & Liyanage, 2021; H. Zhang et al., 2017; S. Zhang, 2019)

Bibilography:

Abdellatif, A. A., Abo-Eleneen, A., Mohamed, A., Erbad, A., Navkar, N. V, & Guizani, M. (2023). Intelligent-slicing: An AI-assisted network slicing framework for 5G-and-beyond networks. *IEEE Transactions on Network and Service Management*, *20*(2), 1024–1039.

Jiang, M., Condoluci, M., & Mahmoodi, T. (2017). Network slicing in 5G: An auction-based model. *2017 IEEE International Conference on Communications (ICC)*, 1–6.

Rafique, W., Barai, J. R., Fapojuwo, A. O., & Krishnamurthy, D. (2024). A survey on beyond 5g network slicing for smart cities applications. *IEEE Communications Surveys & Tutorials*, *27*(1), 595–628.

Rost, P., Mannweiler, C., Michalopoulos, D. S., Sartori, C., Sciancalepore, V., Sastry, N., Holland, O., Tayade, S., Han, B., & Bega, D. (2017). Network slicing to enable scalability and flexibility in 5G mobile networks. *IEEE Communications Magazine*, *55*(5), 72–79.

Subedi, P., Alsadoon, A., Prasad, P. W. C., Rehman, S., Giweli, N., Imran, M., & Arif, S. (2021). Network slicing: A next generation 5G perspective. *EURASIP Journal on Wireless Communications and Networking*, *2021*(1), 102.

Wijethilaka, S., & Liyanage, M. (2021). Survey on network slicing for Internet of Things realization in 5G networks. *IEEE Communications Surveys & Tutorials*, *23*(2), 957–994.

Zhang, H., Liu, N., Chu, X., Long, K., Aghvami, A.-H., & Leung, V. C. M. (2017). Network slicing based 5G and future mobile networks: Mobility, resource management, and challenges. *IEEE Communications Magazine*, *55*(8), 138–145.

Zhang, S. (2019). An overview of network slicing for 5G. *IEEE Wireless Communications*, *26*(3), 111–117.

Experiement-5

Aim: Secondary data plotting and analysis of given data set using Origin.

Analysis of Raman Spectra

Introduction:

The Raman effect is the inelastic scattering of photons by molecules, causing a change in the frequency and energy of incident light, usually from a laser.

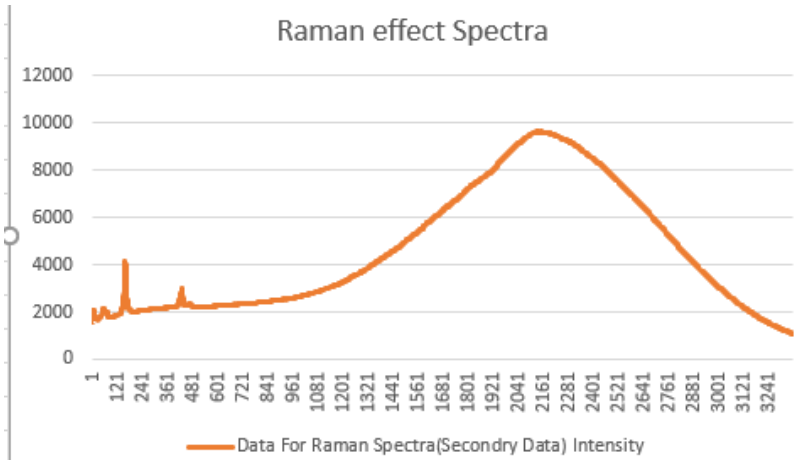
Discovered in 1928, it occurs when photons interact with molecular vibrations, providing unique "fingerprint" spectral data for chemical structure, bond analysis, and material identification in gases, liquids, and solids.

Key Characteristics of the Raman Effect:

- **Inelastic Scattering:** Unlike elastic Rayleigh scattering, Raman scattering involves an exchange of energy between the photons and the molecules.
- **Energy Shifts (Stokes/Anti-Stokes)**: Scattered photons can have lower energy (Stokes lines) or higher energy (anti-Stokes lines) than the incident light.
- **Molecular Fingerprint:** Raman shifts are directly related to the rotational and vibrational energy levels of the specific molecule, acting as a unique identification method.
- **Polarization:** Raman scattered light is often strongly polarized.

The Raman effect is extensively used in chemical analysis, identifying substances (such as pharmaceutical, forensic, and art samples), and determining structural information for both organic and inorganic materials.

Data Plotting



Overall Shape of the Spectrum

The spectrum shows a gradual rise → single dominant peak → gradual decay

Interpretation:

- This is not a multi-peak molecular fingerprint spectrum
- It looks like a band-dominated spectrum, not discrete vibrational lines

Nature of the Peak

- The peak is clearly broad

This strongly suggests:

- Amorphous or disordered material, not crystalline

Lack of Multiple Peaks

- No significant secondary sharp peaks

Implications:

- Not a complex organic molecule (they show many peaks)
- Not a crystalline lattice
- Likely:
    - simple bonding environment, or
    - overlapping modes forming a band

Symmetry of the Curve

- Smooth rise and fall → fairly symmetric

Suggests:

- A single dominant vibrational mode
- Minimal interference from other modes

Material Level Interpretation

- Material is likely:
    - amorphous or semi-ordered
    - Possibly polymeric or carbon-based

- Spectrum dominated by:
    - One dominant vibrational band

Inference

The spectral characteristics suggest that the material is likely amorphous or possesses a low degree of crystallinity. The dominance of a single broad peak indicates a relatively simple or repetitive bonding environment, which is commonly observed in polymeric or disordered carbon-based materials. While the peak position may correspond to regions typically associated with specific bond vibrations, the absence of proper axis calibration prevents definitive assignment.

Analysis of XRD

Introduction:

X-ray Diffraction (XRD) is a powerful technique used to analyze the crystal structure of materials. It is based on the constructive interference of monochromatic X-rays scattered by atoms in a crystalline lattice.

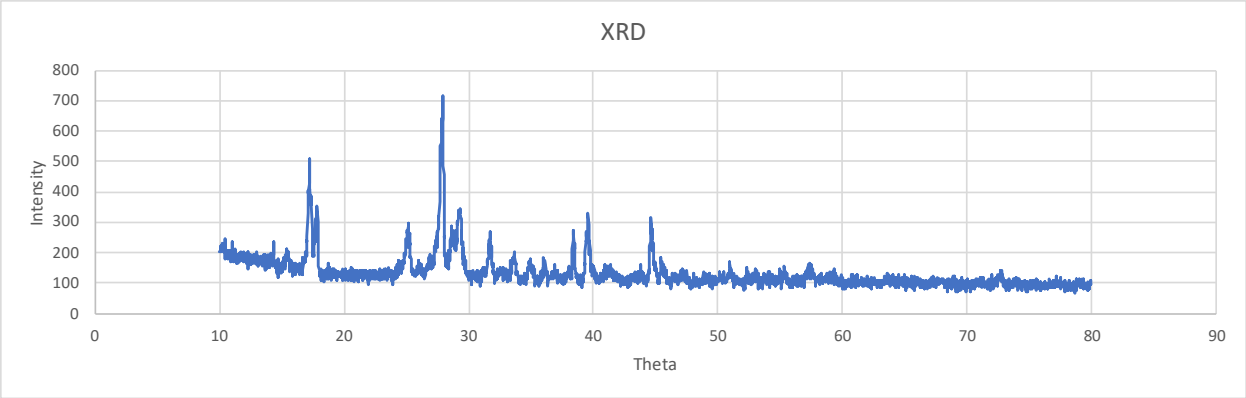
Discovered by Max von Laue in 1912, XRD provides information about lattice spacing, phase identification, and degree of crystallinity of materials by analyzing diffraction patterns obtained as intensity vs diffraction angle (2θ).

Key Characteristics of XRD:

• Bragg's Law: Diffraction occurs when X-rays satisfy the condition $n\lambda = 2d \sin\theta$, relating wavelength (λ), interplanar spacing (d), and angle (θ).
• Phase Identification: Each crystalline material produces a unique diffraction pattern.
• Crystallinity: Sharp peaks indicate crystalline nature, while broad humps indicate amorphous structure.
• Peak Position & Intensity: Provide information about atomic arrangement and preferred orientation.

XRD is widely used in material science for identifying unknown materials, studying crystal structure, and analyzing phase composition in metals, ceramics, polymers, and nanomaterials.

Data Plotting



Overall Shape of the Spectrum

The spectrum shows multiple sharp peaks with one dominant peak around ~27–28°, followed by several smaller peaks and gradual noise background.

Interpretation:

• This is clearly a multi-peak diffraction pattern
• Indicates presence of crystalline phases (not amorphous)
• The pattern is not perfectly clean, suggesting structural imperfections

Nature of the Peaks

• Peaks are sharp but irregular in intensity

This strongly suggests:

• Crystalline or polycrystalline material
• Not a highly ordered single crystal
• Presence of defects or disorder

Presence of Multiple Peaks

• Several peaks observed between 20°–50°

Implications:

• Material has a defined crystal structure
• Not a simple amorphous substance
• Likely:
  Polycrystalline material
  Possible mixed or impure phase system

Dominant Peak Analysis

• A strong peak is observed near 27–28° (2θ)

Suggests:

• Possible correspondence to (002) plane of graphitic carbon
• Could also resemble rutile TiO_2 peak, but lacks full matching pattern

Peak Distribution & Pattern Matching

• Absence of clearly structured secondary peaks (as seen in TiO_2)
• Peaks appear non-uniform and less systematic

Suggests:

• Not a well-defined metal oxide like TiO_2
• More consistent with:
o Graphitic or carbon-based material
o Partially ordered (semi-crystalline) structure

Background and Noise

• Noticeable noise and baseline fluctuation

Indicates:

• Structural disorder
• Possible impurities
• Experimental or sample preparation limitations

Material Level Interpretation

- Material is likely:
  o Semi-crystalline or polycrystalline
  o Carbon-based (graphitic-like)
  o Contains defects or disordered regions

• Structure dominated by:
o One major crystalline plane (~27–28°)
o Additional weak diffraction peaks

Inference

The XRD pattern shows multiple sharp peaks confirming the crystalline nature of the material. The presence of a dominant peak near 27–28° suggests a strong similarity with graphitic carbon structures, particularly the (002) plane. However, the irregular peak distribution and background noise indicate partial disorder or defects, suggesting the material is semi-crystalline rather than highly ordered.

Although the main peak position is close to that of rutile TiO_2, the absence of its characteristic secondary peaks makes TiO_2 identification less likely. Therefore, the material is most reasonably identified as defect-rich or partially crystalline graphitic carbon.

Experiment-6

Aim:- Applications of AI driven apps like Litmap, explain paper, open access plagiarism tools for research work

Theory:-

Literature mapping (or "Litmapping") is a method of visualizing the relationships between academic papers based on citation networks rather than just keywords.

·**Seed Mapping:** A "Seed" paper (a foundational article in your field) is used as a starting point. The AI analyzes its references and the papers that have cited it to create a visual graph.

·**Discovery of Research Gaps:** By visualizing clusters of research, students can identify "white spaces" or gaps where little research has been conducted, helping to refine a thesis.

·**Citation Trajectory:** It tracks the evolution of an idea over time, showing how an initial discovery in electronics (e.g., a specific MOSFET design) evolved through subsequent iterations.

Experiment-6

Aim

Applications of AI-driven tools like Litmaps, Explainpaper, and plagiarism tools in research work.

Theory

AI-driven tools are increasingly used in research to simplify literature review, understanding, and originality checking.

Litmaps helps in literature mapping by visualizing relationships between research papers using citation networks. A seed paper is selected, and related papers are connected through citations, helping identify research trends and gaps.

Explainpaper helps in understanding complex research papers by providing simple explanations of highlighted text. It reduces the difficulty of reading technical content.

Plagiarism tools such as Paperpal help in checking similarity and ensuring originality in research work. These tools highlight copied content and provide a similarity percentage.

Screenshots and Captions

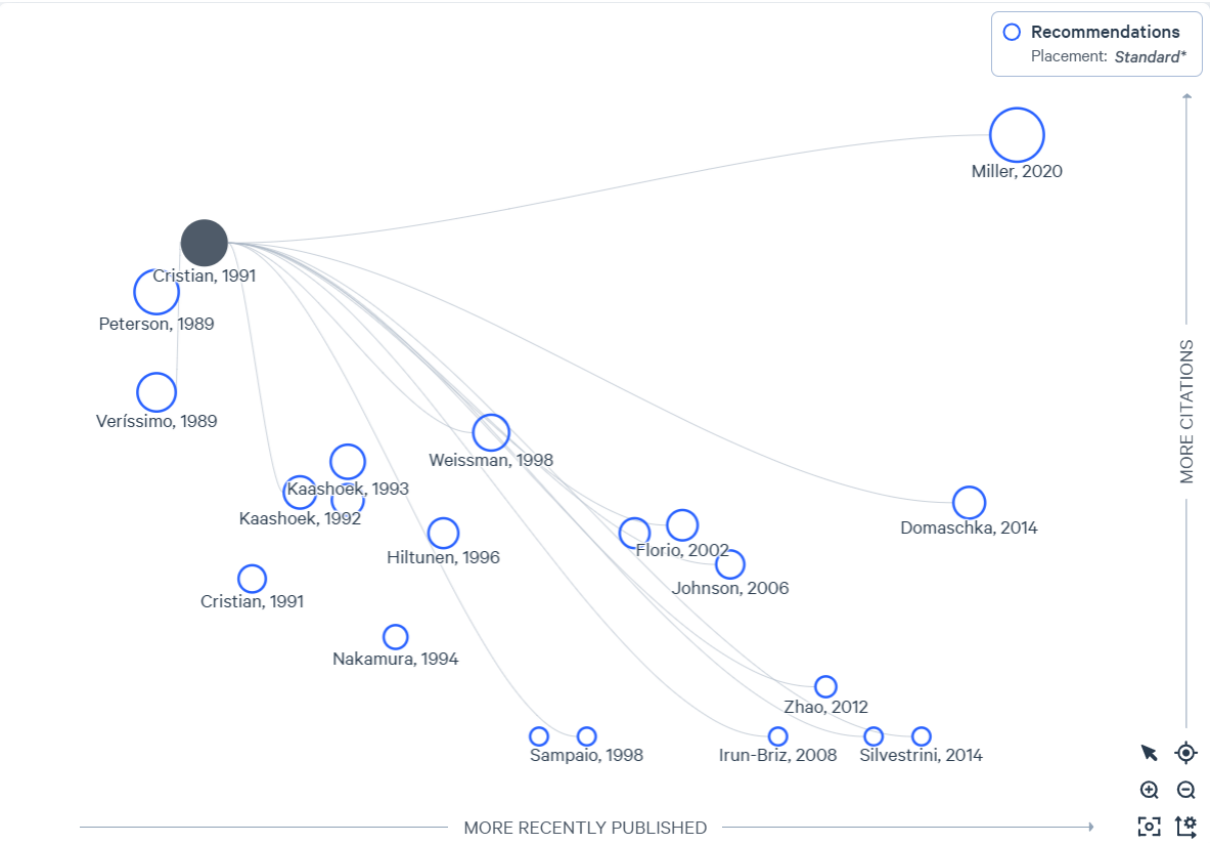


Fig. 1: Litmaps Citation Network

The graph shows connections between research papers. The selected seed paper is linked to related studies, helping in identifying important research and gaps.

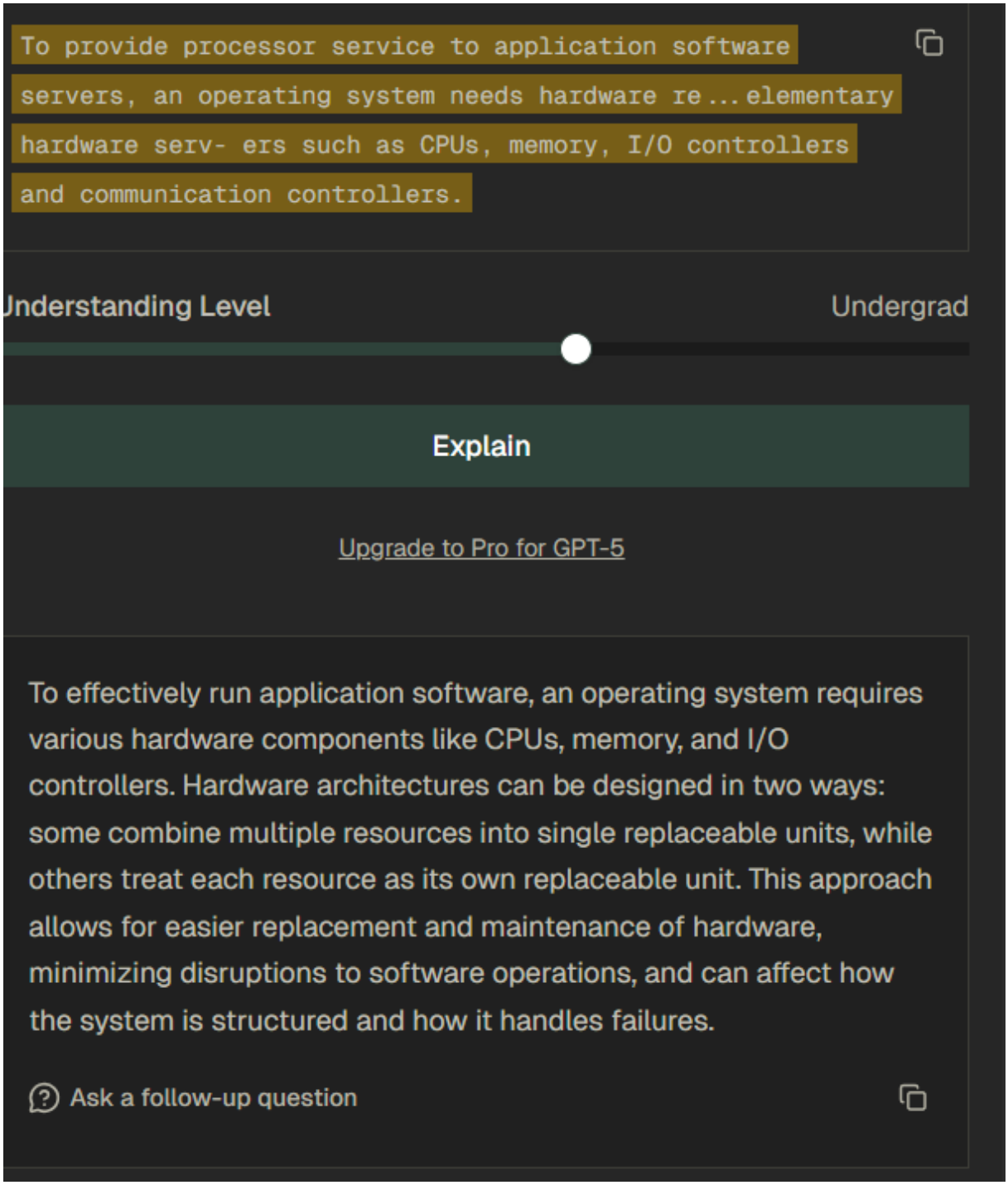


Fig. 2: Explainpaper Output

The highlighted text is explained in simple language, making complex concepts easier to understand.

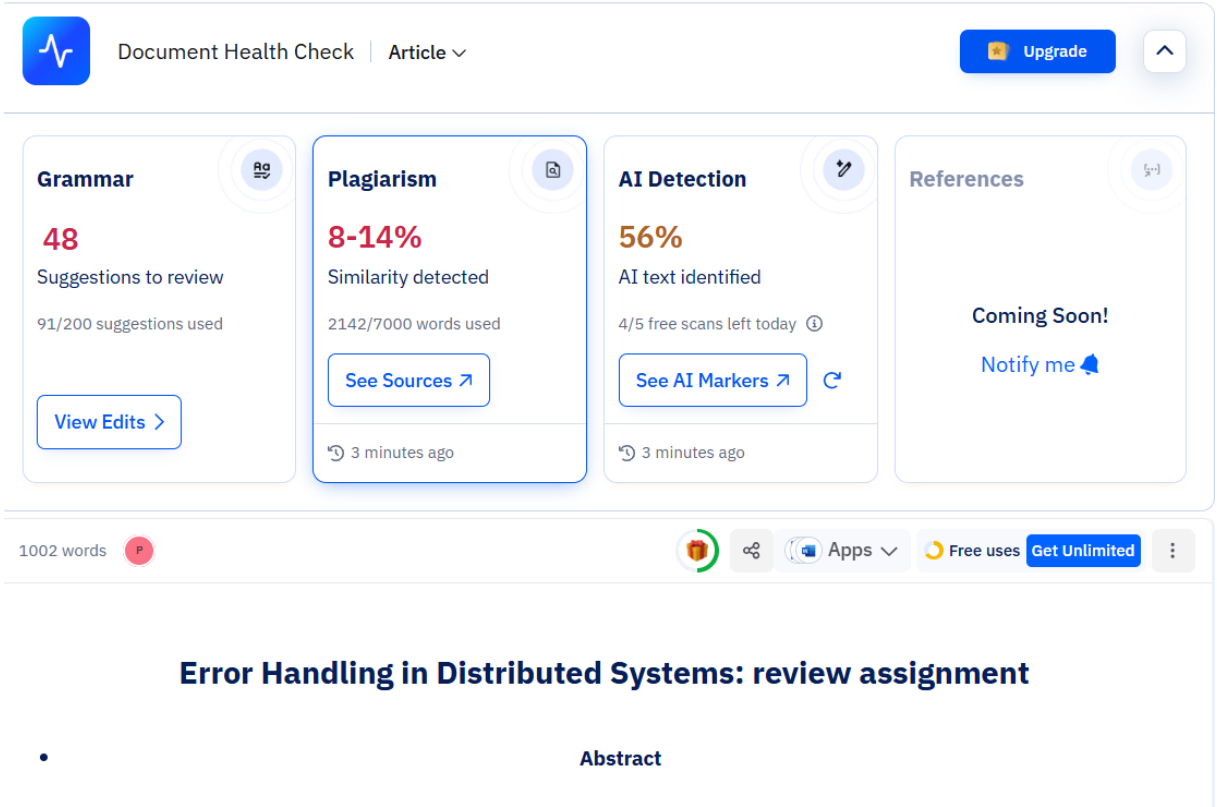


Fig. 3: Plagiarism Check Report

The report shows similarity percentage and highlights matching content to ensure originality.

Practical Application

In this practical, Litmaps was used to find related research papers, Explainpaper was used to understand complex concepts from a research paper, and Paperpal was used to check plagiarism and ensure originality of the report.

Observation

- Litmaps helped in identifying relevant research papers quickly.
- Explainpaper simplified difficult academic content.
- Plagiarism tool ensured originality and proper writing.

Result

AI-driven tools improved the efficiency of research work by helping in literature review, understanding research papers, and checking plagiarism.

Conclusion

AI-based tools play an important role in modern research by making the process faster, easier, and more reliable. These tools help students and researchers perform better quality work with less effort.

Experiment-7

Aim:- Writing a Review Article

Error Handling in Distributed Systems: review assignment

Abstract

As distributed systems underpin the majority of modern digital infrastructure, the ability to handle partial failures is a fundamental requirement. This review article synthesizes current research regarding error handling within distributed environments. We explore foundational failure models, analyze the ”Fallacies of Distributed Computing,” and evaluate the evolution of consensus protocols like Paxos and Raft. Furthermore, we examine modern architectural resilience patterns, the role of failure detectors, and the emerging influence of Machine Learning in proactive fault prediction. The study concludes that the future of distributed error handling lies in autonomous ”self-healing” systems that balance the trade-offs between consistency and availability.

Introduction

Distributed systems consist of multiple autonomous components that communicate via message passing over potentially unreliable networks [1]. Unlike monolithic architectures, where a failure is typically binary, distributed systems are defined by partial failures. A specific node might crash, a network switch might drop packets, or a database shard might become unreachable while the rest of the system remains functional [2].

The primary challenge in error handling is maintaining a consistent global state despite these localized disruptions. This difficulty is encapsulated by the CAP theorem, which states that during a network partition, a system must choose between strict consistency (C) and high availability (A) [3]. This review explores the mechanisms used to navigate these trade-offs and build resilient software.

The Fallacies of Distributed Computing

To handle errors effectively, one must first recognize the underlying causes of system fragility. Peter Deutsch famously identified eight ”Fallacies of Distributed Computing” that often lead to poor error-handling logic:

1. The network is reliable.
2. Latency is zero.
3. Bandwidth is infinite.
4. The network is secure.
5. Topology doesn't change.
6. There is one administrator.
7. Transport cost is zero.
8. The network is homogeneous.

Modern error handling assumes the opposite of these fallacies, treating every network call as a potential point of failure that must be handled gracefully.

Taxonomy of Failure Models

Distributed system designers use formal failure models to define the assumptions under which their logic must operate.

Crash and Omission Models

The **Crash-StopModel** assumes a node stops functioning and never recovers. The **Crash-RecoveryModel** is more realistic, allowing nodes to resume operation after a failure, often relying on persistent logs to restore state. The **Omission Model** focuses on transient network failures where messages are lost or dropped due to buffer overflows.

Byzantine and Arbitrary Failures

The most severe category is the **Byzantine Model**, where nodes may send conflicting or malicious data to different parts of the system [4]. Handling such errors requires specialized protocols like Practical Byzantine Fault Tolerance (PBFT), which are critical for high-security environments and blockchain systems [10, 13].

Consensus: Agreement in the Presence of Faults

If a node fails, the operational members of the cluster must agree on the system's current state to maintain safety and liveness.

The FLP Impossibility Result

A cornerstone of distributed theory is the FLP result (Fischer, Lynch, and Paterson), which proves that in a fully asynchronous system, it is impossible to reach guaranteed consensus if even one node crashes [9]. This forces practical systems to use timeouts or randomized algorithms to make progress.

Raft and Paxos Protocols

Paxos was the industry standard for decades but was notoriously difficult to implement. Raft was introduced to provide an ”understandable” alternative, breaking error recovery into two distinct phases: leader election and log replication [5].

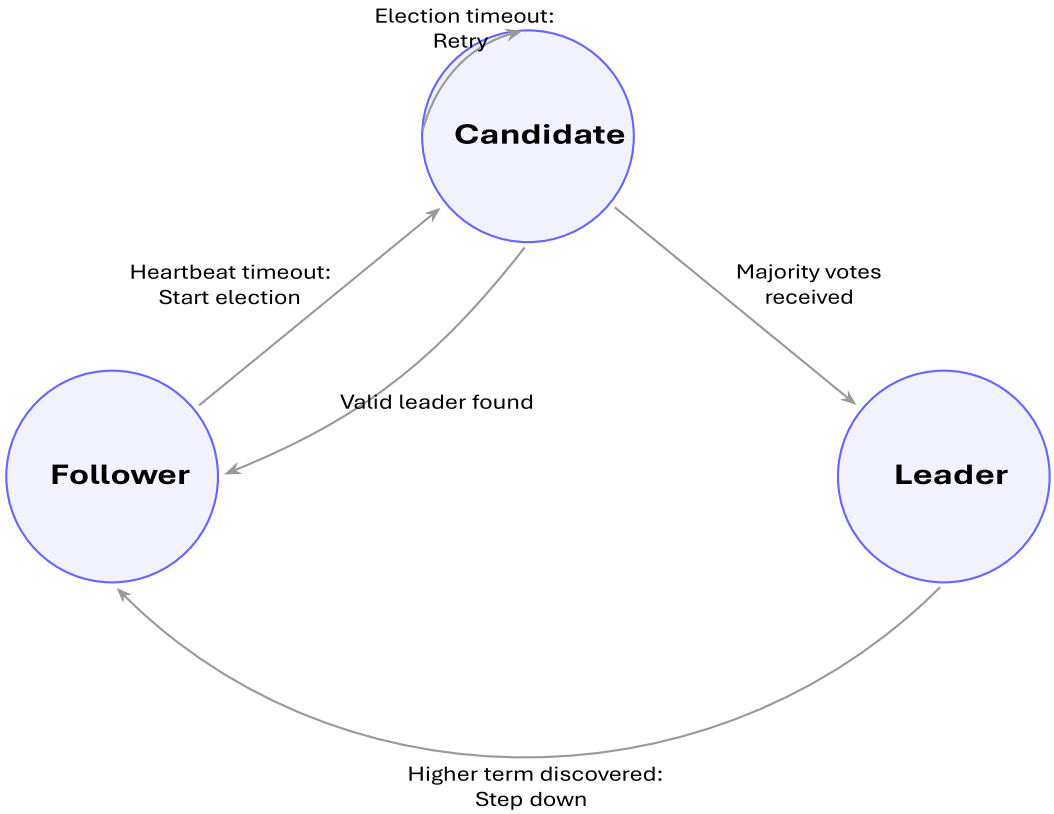


Figure 1: The Raft Consensus state machine. This diagram illustrates how nodes transition between states to ensure system safety and liveness during node failures [5].

Modern Resilience Patterns

In microservice architectures, error handling has shifted from low-level network logic to high-level architectural patterns.

Circuit Breakers and Bulkheads

The **Circuit Breaker** pattern prevents a system from repeatedly trying to execute an operation that is likely to fail [7]. The **Bulkhead** pattern isolates resources (such as thread pools) so that a failure in one component does not exhaust the resources of the entire system.

The Saga Pattern

For distributed transactions, the **Saga Pattern** replaces traditional atomic commits. It breaks a large transaction into smaller steps. If an error occurs, the system executes ”compensating transactions” to undo earlier successful steps [11].

Recovery and Proactive Fault Tolerance

Modern systems move from reactive recovery to proactive fault prediction.

Checkpointing and Rollback

Coordinated checkpointing allows processes to save a consistent global state. In the event of a crash, processes roll back to the last checkpoint [6].

Predictive Maintenance with AI

Machine Learning models analyze telemetry and logs to identify patterns preceding hardware failure. Systems can then pre-emptively migrate tasks to healthy nodes before a failure occurs [8].

Conclusion

Error handling in distributed systems is a multifaceted discipline. While foundational protocols like Raft ensure consistency, modern patterns like Sagas manage application-level complexity. As systems continue to scale, the integration of AIdriven predictive maintenance and Chaos Engineering will define the next generation of self-healing distributed architectures.

References

[1] Coulouris G, Dollimore J, Kindberg T and Blair G 2012 *Distributed Systems: Concepts and Design* (Addison-Wesley)

[2] Lamport L 2001 Paxos Made Simple *ACM SIGACT News* **32**(4) 18–25

[3] Brewer E 2012 CAP twelve years later: How the ”rules” have changed *Computer* **45**(2) 23–29

[4] Lamport L, Shostak R and Pease M 1982 The Byzantine Generals Problem *ACM TOPLAS* **4**(3) 382–401

[5] Ongaro D and Ousterhout J 2014 In Search of an Understandable Consensus Algorithm *Proc. USENIX ATC* 305–319

[6] Elnozahy E N, Alvisi L, Wang Y M and Johnson D B 2002 A survey of rollbackrecovery protocols *ACM Comput. Surv.* **34**(3) 375–408

[7] Nygard M 2018 *Release It! Design and Deploy Production-Ready Software* (Pragmatic Bookshelf)

[8] Assiri B and Sheneamer A 2025 Fault tolerance in distributed systems using deep learning approaches *PLOS ONE* **20**(1)

[9] Fischer M J, Lynch N A and Paterson M S 1985 Impossibility of distributed consensus with one faulty process *J. ACM* **32**(2) 374–382

[10] Castro M and Liskov B 1999 Practical Byzantine Fault Tolerance *Proc. OSDI* 173–186

[11] Helland P 2007 Life beyond Distributed Transactions *CIDR* 132–141

[12] Vogels W 2009 Eventually Consistent *Commun. ACM* **52**(1) 40–44

[13] Veronese G S et al 2013 Efficient Byzantine Fault-Tolerance *IEEE Trans. Comput.* **62**(1) 161–174